TRABALHO PRÁTICO 04 — AGENTES INTELIGENTES

GCC128 – Inteligência Artificial — Universidade Federal de Lavras
Pyêtro Augusto Malaquias e Matheus Gomes Monteiro

1. Descrição da aplicação

Desenvolveu-se um **assistente de planejamento de viagens** composto por uma equipe de agentes inteligentes, implementado em **LangGraph** com o modelo gratuito `openai/gpt-oss-20b` servido pelo Groq. O usuário descreve a viagem pretendida e a equipe levanta as informações necessárias, devolvendo uma recomendação. A aplicação conta com quatro agentes especialistas — pesquisador, meteorologista, orçamentista e redator — e três ferramentas declaradas com o decorator `@tool`: `buscar_destino`, que consulta a Wikipédia em português; `consultar_clima`, que consulta a API pública Open-Meteo; e `calcular`, que estima custos. Nenhuma delas exige chave de acesso.

O aspecto central do projeto é que **a ordem de atuação dos agentes não está escrita no código**. Um agente supervisor, também baseado em LLM, lê o pedido a cada rodada e decide qual especialista deve agir, ou encerra a execução. Cada especialista recebe apenas as ferramentas do seu domínio, de modo que o meteorologista não enxerga a calculadora nem o orçamentista a Wikipédia, o que reduz a chance de escolha equivocada de ferramenta. O redator é o único sem ferramenta: cabe a ele consolidar as contribuições dos colegas.

2. Desenho dos agentes

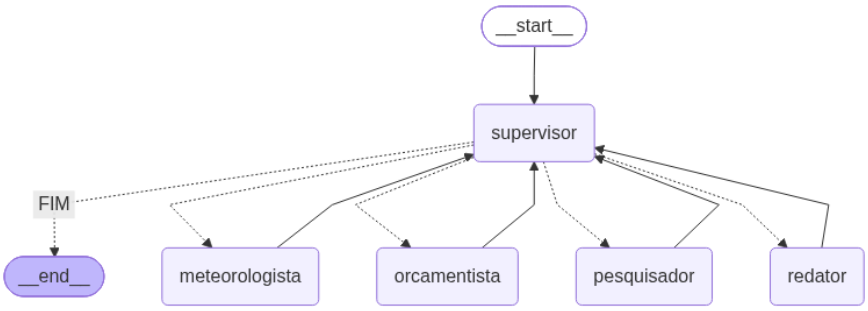

Figura 1 — Grafo da equipe, gerado pelo próprio LangGraph durante a execução. As arestas tracejadas partem do supervisor e representam a rota decidida em tempo de execução; as contínuas devolvem o controle a ele.

3. Resultados

Pedido do usuário	Rota escolhida pelo supervisor	Agentes
"O que tem para ver em Ouro Preto?"	pesquisador → FIM	1
"Vou a Tiradentes amanhã. Preciso levar guarda-chuva?"	meteorologista → FIM	1
"O que visitar, como estará o tempo e quanto gasto de combustível?"	pesquisador → meteorologista → orçamentista → redator → FIM	4

Tabela 1 — Rotas percorridas em três pedidos distintos, sem qualquer alteração no código entre as execuções.

Os três pedidos percorreram rotas diferentes. No primeiro, apenas o pesquisador foi acionado; a consulta de clima sequer chegou a ser feita, por não ser necessária. No segundo, somente o meteorologista atuou. No terceiro, que reúne as três demandas, o supervisor acionou os três especialistas e, ao final, o redator, que consolidou as contribuições em uma recomendação única. O orçamentista devolveu R$ 162,67 para o combustível, valor obtido pela ferramenta a partir de 320 km, 12 km/l e R$ 6,10 por litro.

4. Conclusões

A colaboração entre os agentes cumpriu seu propósito. Cada especialista operou dentro de um papel estreito e o supervisor compôs as contribuições conforme a necessidade de cada pedido. A variação observada nas rotas é precisamente o que distingue um sistema de agentes de um *pipeline*: neste último, a sequência está fixa no código e seria executada integralmente mesmo quando desnecessária, como se verificou na implementação comparativa incluída no notebook.

As ferramentas são o que ancoram o agente na realidade. O modelo de linguagem não executa nada: devolve um pedido estruturado indicando qual função chamar e com quais argumentos, cabendo ao código executá-la. A relevância desse ponto ficou evidente em uma execução preliminar, na qual o pesquisador, sem restrição de escopo, respondeu também sobre clima e custo e calculou o combustível mentalmente, chegando a R$ 162,70 — valor incorreto, frente aos R$ 162,67 que a ferramenta retorna.

Delimitar o escopo de cada agente mostrou-se tão necessário quanto atribuí-lo. Como a resposta daquele especialista aparentava cobrir todo o pedido, o supervisor encerrou a execução prematuramente. A correção envolveu duas frentes: restringir cada `system_prompt` com proibições explícitas e reescrever a instrução do supervisor para avaliar o pedido parte por parte, considerando atendida apenas a porção coberta pelo especialista do domínio correspondente. O episódio ilustra uma característica do sistema: o modelo tende a responder tudo o que lhe é perguntado, ainda que fora de seu papel, e em um arranjo multiagente isso compromete a divisão de trabalho — manifestando-se não como erro de execução, mas como resposta plausível e incorreta.

Limitações. A consulta meteorológica cobre apenas os três dias seguintes à execução, de modo que um pedido referente ao fim de semana pode receber a previsão dos dias imediatamente posteriores. A aplicação tampouco guarda memória entre execuções, o que um `checkpointer` com `thread_id` resolveria, nem solicita aprovação humana antes de ações sensíveis — recurso que o `interrupt` do LangGraph oferece e que seria indispensável caso alguma ferramenta efetivamente realizasse reservas ou compras.

Vídeo de apresentação: https://drive.google.com/file/d/14WK4dGtX3gaEYEWBiOukdkI6LJMBDrY7/view?usp=sharing